

Where Training Data Comes From

Quantity, quality, and copies · video 2:18

This lesson answers one question: where does training text come from, and does its quality matter? Large models train on trillions of tokens, mostly from the filtered public web. Two experiments show why filtering matters: mixing junk into the training data raises the loss on clean text (1.644 → 1.776 with half junk), and a passage repeated 1,000 times is memorized word for word, the main reason training data is deduplicated.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 5).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. Quantity and sources	▶ 0:08–0:51	2	☐
2. Quality: junk in the data	▶ 0:51–1:10	1	☐
3. Copies and memorization	▶ 1:10–2:09	3	☐

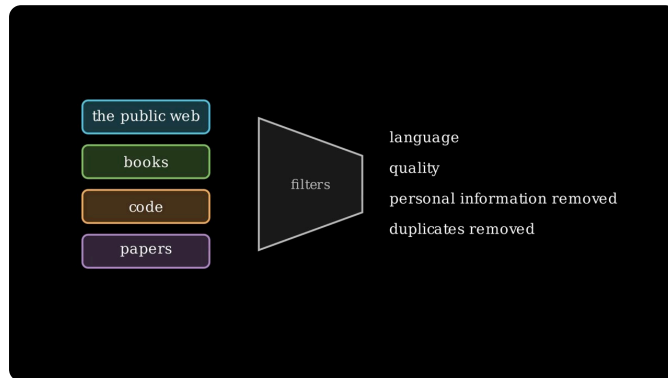
Before you start: Deep Dive episodes 1 (tokenization) and 17 (the loss). Code: `code/d24_training_data` (GPT-2's tokenizer plus seven tiny models, about 35 minutes on a CPU).

CONCEPT 1 OF 3

Quantity and sources

▶ Video 0:08–0:51

Our Tiny Shakespeare is 1,115,394 characters, **338,025** GPT-2 tokens. Meta says Llama 3 was trained on more than **15 trillion** tokens. That much text comes mostly from the public web, crawled at enormous scale, plus books, code and papers; then it is filtered hard: by language, by quality, by removing personal information, and by removing duplicates.



▶ 0:36 Sources (web, books, code, papers) and filters (language, quality, personal information, duplicates).

Tiny Shakespeare: 23% of its 32,777 lines appear more than once

GLOUCESTER:	× 229
DUKE VINCENTIO:	× 193
ROMEO:	× 163
MENENIUS:	× 162

on the web: boilerplate, mirrors, copies of copies

▶ 0:50 Even Tiny Shakespeare repeats itself: 23% of its lines appear more than once.

Duplicates are everywhere. Even in our small file, 7,388 of the 32,777 non-empty lines (**23%**) appear more than once, mostly speaker names: “GLOUCESTER:” 229 times, “DUKE VINCENTIO:” 193. On the web it is boilerplate, mirrors, and copies of copies.

KEY IDEA

Raw text is plentiful; good, unique text is what filtering is for.

Check 1: Quantity and sources

Answer all 2 questions before looking at the answers. Pass mark: all correct.

1.1 CALCULATION

How many times larger than Tiny Shakespeare (in GPT-2 tokens) is a 15-trillion-token dataset?

.....

1.2 CALCULATION

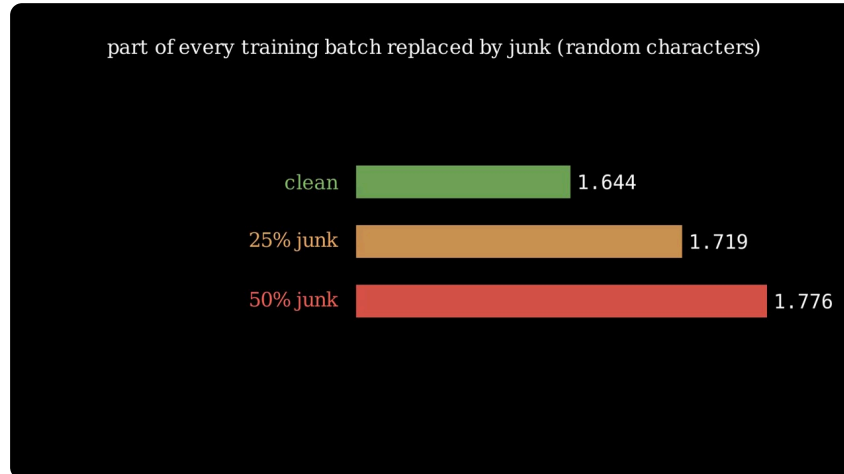
About how many characters per GPT-2 token does Tiny Shakespeare have?

.....

Quality: junk in the data

▶ Video 0:51-1:10

Part of every training batch is replaced by junk (random characters), and the loss is measured on clean validation text after 2,000 steps: clean data **1.644**, a quarter junk **1.719**, half junk **1.776**. Junk wastes compute (those sequences teach nothing useful) and it hurts the model.



▶ 1:09 Clean 1.644; 25% junk 1.719; 50% junk 1.776 (loss on clean validation text).

KEY IDEA

Every bad token costs training time and pulls the model away from the text you care about.

Check 2: Quality: junk in the data

Answer all 1 questions before looking at the answers. Pass mark: all correct.

2.1 SHORT ANSWER

With 50% junk, the model sees half as much real text in the same number of steps. How much of the damage could that alone explain?

.....

CONCEPT 3 OF 3

Copies and memorization

▶ Video 1:10-2:09

One 65-character passage from the validation text, which the model normally never sees, is slipped into training again and again. Never seen: loss **1.558** on the passage, next character right **48%** of the time. 200 copies: 0.467, 94% right, but started from its first 20 characters the model drifts off after one mistake (2 of 45 characters right).

1,000 copies: loss 0.062, 100% right, and from the first 20 characters it writes the next **45 exactly**. Meanwhile the loss on everything else barely moves (1.644 → 1.657). The model has memorized one passage word for word: repeated text wastes training, and what a model memorizes it can repeat, private details or copyrighted pages. That is why data is deduplicated.

copies	passage loss	next char right	writes from 20 chars
0	1.558	48%	4 / 45
200	0.467	94%	2 / 45

true: each her fingering; When, with a most impati
wrote: he storm of the strike of the storm...KING RI

copies	passage loss	next char right	writes from 20 chars
0	1.558	48%	4 / 45
200	0.467	94%	2 / 45
1,000	0.062	100%	45 / 45
2,000	0.033	100%	45 / 45

wrote: each her fingering; When, with a most impatie (word for word)

loss on everything else: 1.644 → 1.657 (barely moves)

▶ 1:37 Never seen: 48% of next characters right; 200 copies: 94%, but generation drifts after one mistake.

▶ 1:51 1,000 copies: 100%, and the next 45 characters written word for word; other text barely changes.

KEY IDEA

Repetition turns prediction into memorization; deduplication prevents it.

Check 3: Copies and memorization

Answer all 3 questions before looking at the answers. Pass mark: all correct.

3.1 PUT IN ORDER

Order by how well the passage was memorized: *200 copies* · *never* · *1,000 copies*.

3.2 TRUE OR FALSE

"A 94% next-character accuracy guarantees the model can reproduce the passage."

True ☐ False ☐

3.3 HANDS-ON CODE

Try it yourself. In `training_data.py`, change the repetition list to `(5,)` (400 copies). Is the passage reproduced?

```
for every in (5,):  
    ...
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. Quantity and sources · rewatch ▶ 0:08-0:51

1.1 About 44 million times. $15 \times 10^{12} / 338,025 \approx 4.4 \times 10^7$.

1.2 About 3.3. $1,115,394 / 338,025$.

2. Quality: junk in the data · rewatch ▶ 0:51-1:10

2.1 Most of it: in episode 23, halving a similar model's data (4.1M → 2.0M tokens) raised its loss by 0.11 (1.684 → 1.798); here half junk cost 0.13 (1.644 → 1.776). At this scale, junk mainly wastes the steps it occupies; the rest of the gap may be the junk's own effect.

3. Copies and memorization · rewatch ▶ 1:10-2:09

3.1 1,000 copies (100%, 45/45) → 200 copies (94%, 2/45) → never (48%). From the episode's run.

3.2 False. One early mistake sends generation elsewhere: at 200 copies it wrote only 2 of 45 characters right.

3.3 Yes: loss 0.134, 98% of next characters right, and 45/45 written word for word. The threshold lies between 200 and 400 copies for this model and passage. Memorization grows with repetition; the exact threshold depends on the model and the passage.